HGAME 2026 WEEK2

队伍：springbot

id：#0x000b07

问卷

有人发现源代码里就有flag吗（）

hgame{ 1/1

\":\"hgame{see_you_n3xt_ye0r!!!}\",\"link\":\"\"}]}},\"redirectAfterSubmit\":null,\"tableQu

Misc

Invest on Matrix

发现结构

首先得读懂题目，我们把一个25x25的矩阵分为25个1x1的小块，编号1~25，花费是等同于编号数量的pts

25x25的规整图形，我们肯定想到了同样尺寸的二维码，不妨先打开花费最小的1号矩阵

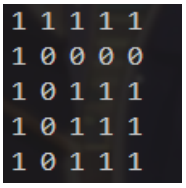
1 1 1 1 1 1 0 0 0 0 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1

只有0和1，似乎和二维码是对应的

我们写个脚本转化成图形看看

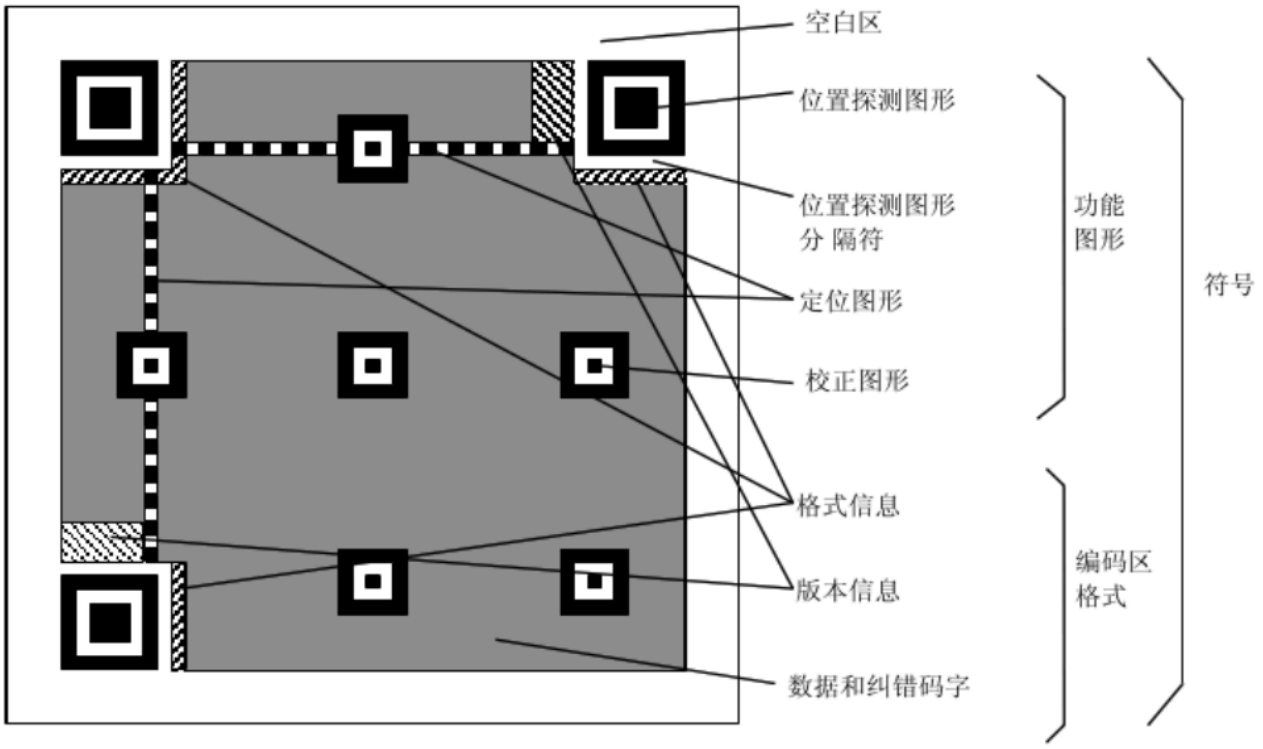
```
s = "1 1 1 1 1 1 0 0 0 0 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1"
s = "".join(s.split())

for i in range(5):
    for j in range(5):
        print(s[i*5+j],end = "")
        print(" ",end = "")
    print()
```

结果如下：



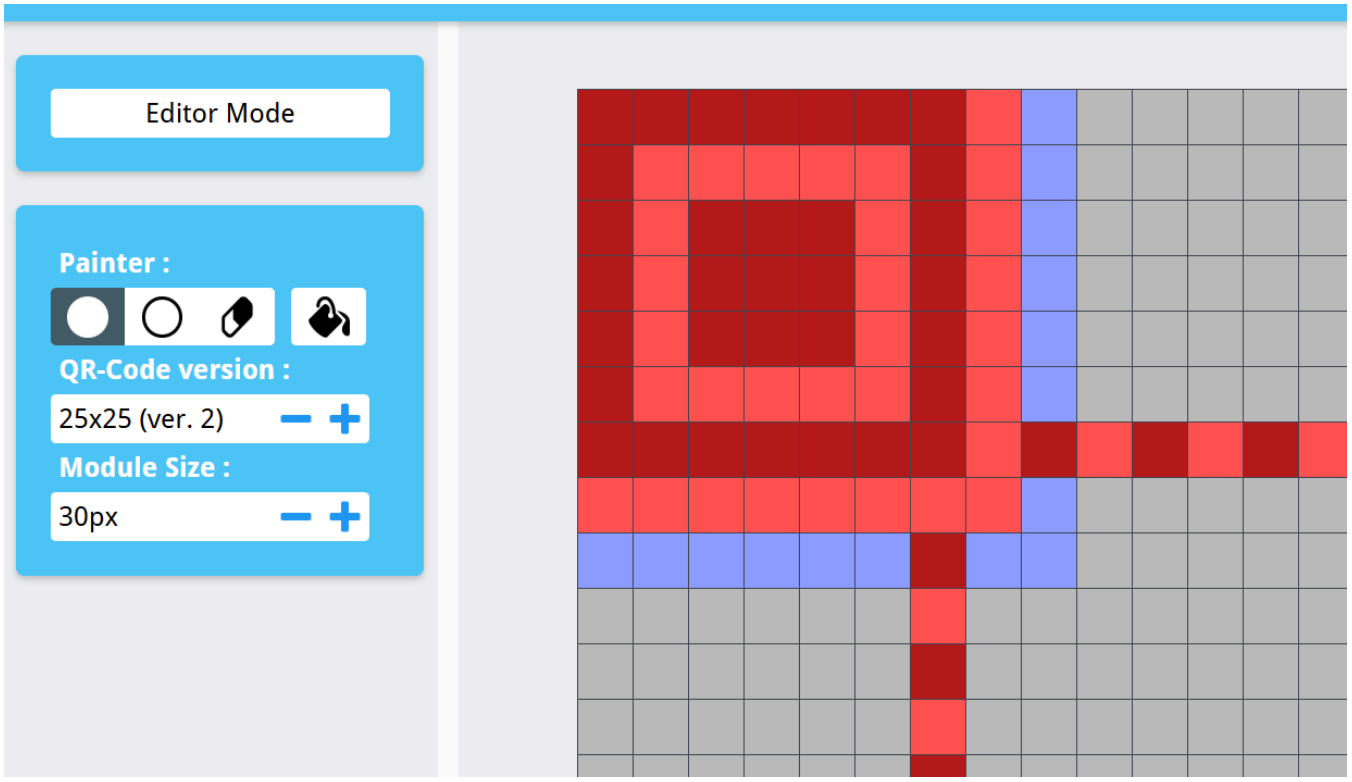
这里我们需要知道的是二维码（QR code）的基本结构（可以参考这个博客学习）

这里借用一下图（谢谢啦）



如果还是不确定，那我们再打开[qrazybox工具](qrazybox工具)看一下

我们知道，左上角的第一号格子就是定位符的左上侧，这个矩阵是完全符合的



如图，调到25x25的version后，我们的左上角的定位符格子是确定的，且和题目给出的矩阵一致

那么我们需要做的就是补全这个二维码。

补全二维码

对于这题，要想做出来flag还是很容易的，就是买下所有的hint，然后转换为二维码解码即可。

但是，我们有更好的策略，即利用二维码的可纠错性（即**Reed-Solomon** 纠错算法）来通过尽量少的方块恢复二维码的信息

这是我解此题的代码，可见并没有开启所有hint

```
import numpy as np
from reedsolo import RSCodec, ReedSolomonError

def final_decode():
    m = np.full((25, 25), -1)
    def f(rs, cs, d):
        for i, v in enumerate([int(x) for x in d.split()]):
            m[rs + i // 5, cs + i % 5] = v

    # 已购买的所有 Hint 数据
    f(0, 0, "1 1 1 1 1 1 0 0 0 0 1 0 1 1 1 1 0 1 1 1 1 0 1 1 1") # 1
    f(0, 5, "1 1 0 1 1 0 1 0 1 1 0 1 0 1 0 0 1 0 0 0 0 1 0 0 1") # 2
    f(0, 10, "0 0 1 0 0 1 1 1 0 1 0 0 0 1 1 0 1 1 0 1 0 1 1 0 0") # 3
    f(0, 15, "0 0 0 1 1 0 0 0 1 0 1 1 0 1 0 1 1 0 1 0 0 0 0 1 0") # 4
    f(0, 20, "1 1 1 1 1 0 0 0 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 1") # 5
    f(5, 15, "1 0 0 1 0 0 1 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 0 1 0") # 9
    f(5, 10, "0 0 1 1 0 1 0 1 0 1 0 1 1 1 1 0 0 1 0 0 0 1 0 0 1") # 8
    f(5, 5, "0 1 0 1 0 1 1 0 1 0 0 0 0 1 0 0 1 0 1 0 1 0 1 0 1") # 7
    f(5, 20, "0 0 0 0 1 1 1 1 1 1 0 0 0 0 0 0 0 1 1 1 0 1 0 1 1") # 10
    f(10, 10, "1 1 0 1 0 0 0 1 0 1 1 1 1 1 0 1 1 0 1 1 1 0 0 1 0") # 13
    f(10, 15, "1 1 1 0 1 0 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 0 0 1 0") # 14
    f(10, 20, "1 1 1 1 1 1 0 0 0 1 1 0 0 0 1 0 0 0 1 0 0 1 0 0 1") # 15
    f(15, 15, "0 1 1 1 1 0 1 1 1 1 0 1 0 0 0 1 1 0 1 0 1 1 0 0 0") # 19
    f(15, 20, "0 0 0 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1") # 20
    f(20, 0, "1 0 1 1 1 1 0 1 1 1 1 0 1 1 1 1 0 0 0 0 1 1 1 1 1") # 21
    f(20, 15, "1 1 1 1 1 0 0 0 1 1 1 0 1 1 1 1 0 1 0 1 0 1 0 0 0") # 24
    f(20, 20, "1 0 0 0 0 0 0 1 0 1 0 0 1 0 1 0 1 0 0 1 0 0 1 1 1") # 25

    # 识别data
    def is_data(r, c):
        if r <= 8 and (c <= 8 or c >= 17): return False
        if r >= 17 and c <= 8: return False
        if r == 6 or c == 6: return False
        if 16 <= r <= 20 and 16 <= c <= 20: return False
        return True

    def get_bits():
        bits = ""
        cols = []
        curr_c = 24
        while curr_c > 0:
            if curr_c == 6: curr_c -= 1
            cols.append((curr_c, curr_c - 1))
            curr_c -= 2
```

```
    for i, (c1, c2) in enumerate(cols):
        rows = range(24, -1, -1) if i % 2 == 0 else range(25)
        for r in rows:
            for c in [c1, c2]:
                if is_data(r, c):
                    val = m[r, c]
                    if val == -1: bits += "?"
                    else:
                        mask = 1 if (c % 3 == 0) else 0
                        bits += str(val ^ mask)
    return bits

raw_bits = get_bits()
codewords = []
erasures_pos = []
# Version 2-M 总共 44 个 Codewords
for i in range(0, 44 * 8, 8):
    byte_s = raw_bits[i:i+8]
    if "?" in byte_s:
        codewords.append(0)
        erasures_pos.append(i // 8)
    else:
        codewords.append(int(byte_s, 2))

print(f"当前未知 Codewords 数量: {len(erasures_pos)}")

rs = RSCodec(16)
try:
    try:
        corrected = rs.decode(bytearray(codewords), erasures_pos=erasures_pos)[0]
    except TypeError:
        corrected = rs.decode(bytearray(codewords))[0]

    full_bits = "".join([format(b, '08b') for b in corrected])
    print("\n--- 成功恢复所有数据块 ---")

    idx = 0
    while idx < len(full_bits) - 4:
        mode = full_bits[idx:idx+4]
        if mode == "0000": break
        if mode == "0100":
            length = int(full_bits[idx+4:idx+12], 2)
            content = ""
            for i in range(length):
                start = idx + 12 + i*8
                content += chr(int(full_bits[start:start+8], 2))
            print(f"【Byte数据段】: {content}")
            idx += 12 + length*8
        else:
            idx += 4
except ReedSolomonError:
    print("纠错仍然失败。")
    # 如果还是失败，显示目前能解析出的原始片段
```

```
        print("尝试解析已知片段:")
        tmp_bits = raw_bits.replace("?", "0") # 暂时填0尝试解析
        mode = tmp_bits[:4]
        if mode == "0100":
            length = int(tmp_bits[4:12], 2)
            res = ""
            for i in range(length):
                b = tmp_bits[12+i*8:12+(i+1)*8]
                res += chr(int(b, 2)) if '0' <= b <= '1' else '?'
            print(f"已知前缀: {res}")

final_decode()
```

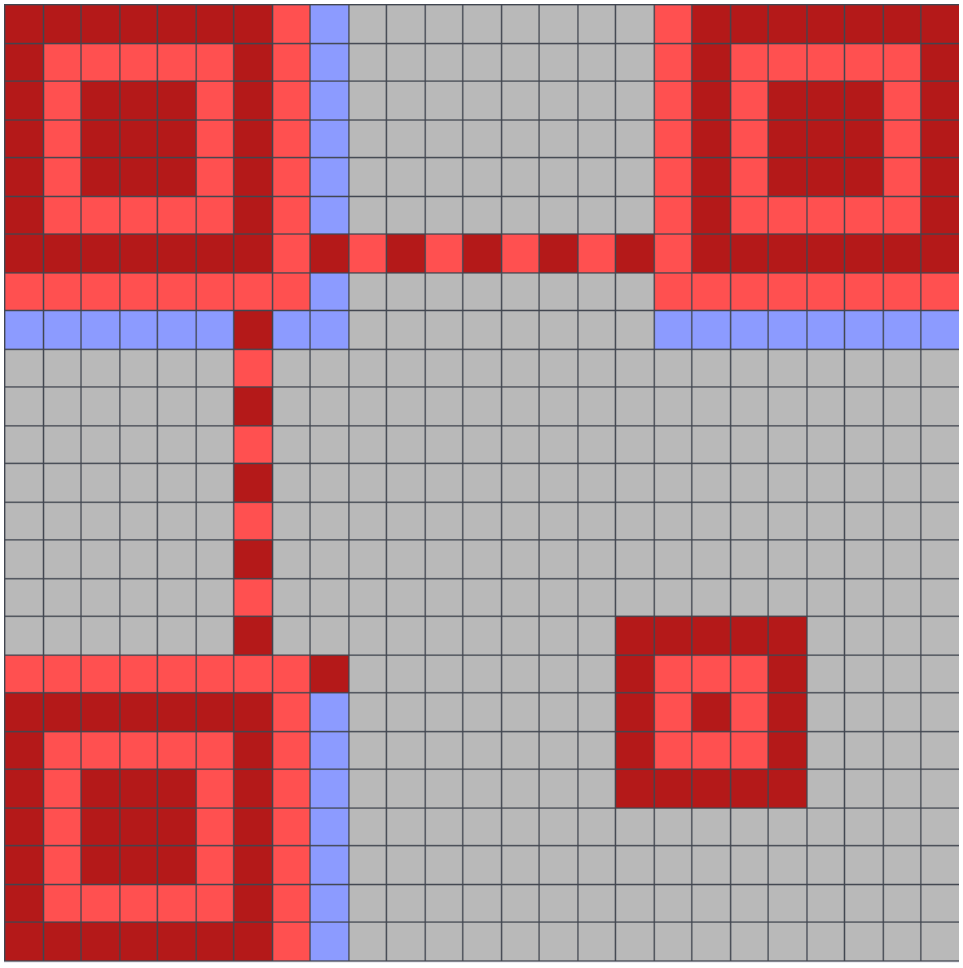
（也可以试试qcrazybox的暴力破解，但是我没成功）

答案是：W0RTH_1T?

优化解法

那么，站在上帝视角看，是否有花费更少pts点数的策略呢？

我们先看一下25x25的结构



首先是三个角的定位区域（即1，5和21，以及旁边的2，4，5，6等等红色部分）我们无须解锁

蓝色部分是格式信息，也无需优先考虑

第二，根据二维码的生成规则，数据是从右下角开始填充，我们应该优先解锁中间和右下部分的含信息区域

与此同时，我们也要知道，越往左和越往上的提示便宜，所以如果能表示相同的信息，优先解锁左边和上方的信息

根据第二条规则，我们先解锁25，24和上面的19，20（根据第三条原则，我们先不解锁22，23）

然后可以再买个15，此时解码的答案是`W0BTH_1T*`，已经很接近了，再添加两个hint加上猜应该可以解出来

我目前尝试的最小值是：10+14+15+19+20+24+25 = 127 pts

`hgame{W0RTH_1T?}`

Vidar Token

看题目大概是一道区块链的信息搜集题

我们打开容器查看，connect Wallet失败，提示"浏览器钱包在非 HTTPS 环境无法直接连接 (> ﹏ <)"

不用着急，我们先打开源码看看

第一层提示：

不用管，继续看app.js

找到connectWallet函数

```
async function connectWallet() {
  walletProvider = null;
  walletStatusEl.classList.remove("active");
  appContentEl.classList.add("locked");
  appContentEl.classList.remove("unlocked");
  vaultStatusEl.textContent = "浏览器钱包在非 HTTPS 环境无法直接连接 (>﹏<)";
}
```

居然是骗我的👀！根本没有连接钱包的逻辑

再往下看，checkEligibility函数通过get方法获取了k.wasm文件，这是一种web端的"汇编语言"，运行速度非常快

（显示：尝试读取元数据）

那我们打开F12，在控制台中直接调用checkEligibility函数

```
checkEligibility();
```

元数据已就绪！我们已经加载了wasm文件

尝试调用tokenURI(0)，看看开头是不是藏了有用的东西

```
(async () => {
    const provider = new ethers.JsonRpcProvider(`${window.location.origin}/rpc`);
    const vault = new ethers.Contract(
      entranceAddress,
      ["function tokenURI(uint256) view returns (string)"],
      provider
    );

    const result = await vault.tokenURI(0);
    if (result.startsWith("data:application/json; base64,")) {
```

```
        const json = atob(result.split("base64,")[1]);
        console.log("【解码后的 JSON】:", json);
    }
})();
```

回显内容如下：

```
【解码后的 JSON】: {"name":"VidarPunks #0","description":"VidarPunks Vault NFT. Seek your fortune with VidarCoin.","attributes":[{"trait_type":"Linked Coin Address","value":"0xc5273abfb36550090095b1edec019216ad21be6c"}],"vidar_coin":"0xc5273abfb36550090095b1edec019216ad21be6c"}
```

我们获得了NFT 的合约地址，下一步就是看看这个地址处有没有什么隐藏的信息。

我们尝试在这个实例中创建并调用可能的函数接口

```
(async () => {
    const coinAddress = "0xc5273abfb36550090095b1edec019216ad21be6c";
    const provider = new ethers.JsonRpcProvider(`${window.location.origin}/rpc`);

    const coin = new ethers.Contract(
        coinAddress,
        [
            "function name() view returns (string)",
            "function symbol() view returns (string)",
            "function flag() view returns (string)",
            "function getFlag() view returns (string)",
            "function secret() view returns (string)",
            "function info() view returns (string)"
        ],
        provider
    );

    try { console.log("Name:", await coin.name()); } catch(e) {}
    try { console.log("Symbol:", await coin.symbol()); } catch(e) {}
    try { console.log("Flag 函数:", await coin.flag()); } catch(e) {}
    try { console.log("getFlag 函数:", await coin.getFlag()); } catch(e) {}
    try { console.log("Secret:", await coin.secret()); } catch(e) {}
    try { console.log("Info:", await coin.info()); } catch(e) {}
})();
```

回显

```
Name: VidarCoin
Symbol:
0x6960606a647c7458356534686d7255344d5e5e6c6f48562a64754258564734502c7d5b5d303635326764607a
```

这个symbol很可疑，我们先from hex，并没有发现异常，继续寻找其他线索

我们在wasm中导出所有功能

```
(async () => {
  const res = await fetch("/wasm/k.wasm");
  const wasm = await res.arrayBuffer();
  const { instance } = await WebAssembly.instantiate(wasm, {});

  console.log("WASM 导出的所有功能:", Object.keys(instance.exports));

})();

// WASM 导出的所有功能: (5) ['memory', 'get_entrance', 'get_basea', 'get_baseb',
'decrypt_logic']
```

get_basea和get_baseb很可以，查看一下函数逻辑

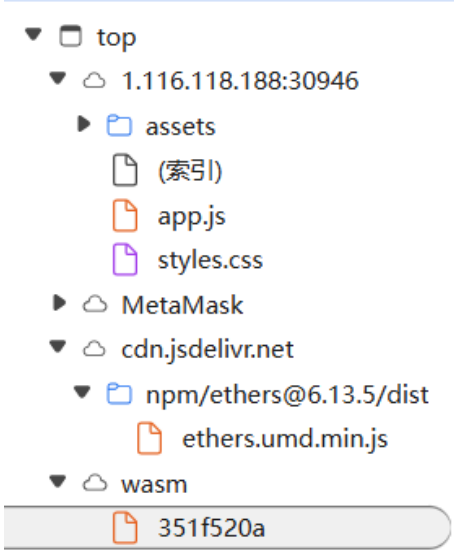
```
(async () => {
  const res = await fetch("/wasm/k.wasm");
  const wasm = await res.arrayBuffer();
  const { instance } = await WebAssembly.instantiate(wasm, {});
  const { memory, get_basea, get_baseb } = instance.exports;

  function readStr(ptr) {
    const bytes = new Uint8Array(memory.buffer, ptr, 100);
    let s = "";
    for (let i = 0; i < bytes.length && bytes[i] !== 0; i++) s +=
String.fromCharCode(bytes[i]);
    return s;
  }

  console.log("Base A 内容:", readStr(get_basea()));
  console.log("Base B 内容:", readStr(get_baseb()));
})();

// BASEA=0x5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d5b5d
// BASEB=0x5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a5a
```

这两个base很明显的看出人为构造的痕迹，那么我们用symbol去和这两个值异或

很像flag了！但是这是flag吗？我们提交发现并不是

接下来有两种方法：

第一种，我们可以在开发者工具-源代码中看到k.wasm的源代码，可以选择直接根据代码逆向，也可以用wasm的分析工具（题目提示的toolkit）去做




```
0x009c        i32.add
0x009d        i32.load8_u
0x00a0        i32.const 90
0x00a3        i32.xor
0x00a4        i32.store8
0x00a7        local.get $i
0x00a9        i32.const 1
0x00ab        i32.add
0x00ac        local.set $i
0x00ae        br $entrance_loop
0x00b0      end $entrance_loop
0x00b1    end $entrance_done
0x00b2    i32.const 0
0x00b4    local.set $i
0x00b6    block $basea_done
0x00b8      loop $basea_loop
0x00ba        local.get $i
0x00bc        i32.const 73
0x00bf        i32.ge_u
0x00c0        br_if $basea_done
0x00c2        i32.const 80
0x00c5        local.get $i
0x00c7        i32.add
0x00c8        i32.const 80
0x00cb        local.get $i
0x00cd        i32.add
0x00ce        i32.load8_u
0x00d1        i32.const 90
0x00d4        i32.xor
0x00d5        i32.store8
0x00d8        local.get $i
0x00da        i32.const 1
0x00dc        i32.add
0x00dd        local.set $i
0x00df        br $basea_loop
0x00e1      end $basea_loop
0x00e2    end $basea_done
0x00e3    i32.const 0
0x00e5    local.set $i
0x00e7    block $baseb_done
0x00e9      loop $baseb_loop
0x00eb        local.get $i
```

不过这里的逻辑似乎有动态生成的部分，静态逆向似乎比较困难

第二种方法就是猜flag

对于hgame{u_4b5oluT3LY_knOW-erC_W@5W-zZZ1145fca}，把leet重新转换为可读的文本

```
4b5oluT3LY -> absolutely
knOW -> know
erC -> erc
W@5W -> WASM （联想wasm）
-zZZ1145fca -> -zZZ114514 （这里带点脑洞，联想114514梗）
```

`hgame{u_absolutely_know-erc_WASM-zZZ114514}`

Crypto

ezRSA

先恢复n和e，再利用LSB Oracle 漏洞解

```
from pwn import *
from Crypto.Util.number import *
import base64

io = remote('1.116.118.188', 31834)

def decrypt(c):
    io.sendlineafter(b"Your choice > ", b"2")
    # 注意这里匹配到换行符，确保缓冲区干净
    io.sendlineafter(b"ciphertext:\n", str(c).encode())
    res = io.recvline().strip()
    return base64.b64decode(res)

def encrypt(p, x):
    io.sendlineafter(b"Your choice > ", b"1")
    io.sendlineafter(b"plaintext:\n", str(p).encode())
    io.sendlineafter(b"flip:\n", str(x).encode())
    res = io.recvline().strip()
    return base64.b64decode(res)

# 1. 准确恢复 n
print("[*] Recovering n...")
m2 = bytes_to_long(decrypt(2))
m4 = bytes_to_long(decrypt(4))
m3 = bytes_to_long(decrypt(3))
m9 = bytes_to_long(decrypt(9))
# n 是 m^2 - (m^2 mod n) 的公约数
n = GCD(m2**2 - m4, m3**2 - m9)

# 如果得到的 n 太大（是 n 的倍数），可以通过除以小素数简化，
# 但通常 GCD(2组) 就能直接得到 n
while n % 2 == 0: n //= 2
print(f"[+] Recovered n: {n}")

# 2. 恢复 e
print("[*] Recovering e...")
# 选择一个足够大的 x (e 只有 50 位)
x_test = 60
c60 = bytes_to_long(encrypt(2, x_test))
# 2^(e ^ 2^60) = 2^(e + 2^60) = 2^e * 2^(2^60)
two_e = (c60 * pow(pow(2, 1 << x_test, n), -1, n)) % n

e = 0
for i in range(50):
```

```
    ci = bytes_to_long(encrypt(2, i))
    # 如果 e 的第 i 位是 0, 则 2^(e ^ 2^i) = 2^e * 2^(2^i)
    if ci == (two_e * pow(2, 1 << i, n)) % n:
        pass
    else:
        e |= (1 << i)
print(f"[+] Recovered e: {e}")

# 3. 获取 Flag 密文并触发 safe=False
io.sendlineafter(b"Your choice > ", b"3")
c_flag = bytes_to_long(base64.b64decode(io.recvline().strip()))
print(f"[+] Got flag ciphertext. Safe mode is now OFF.")

# 4. Last Byte Oracle 攻击
# 原理：disguise 后的最后一个字节是真实的 (m * 256^k % n) % 256
low = 0
high = n
n_inv = pow(n, -1, 256)

print("[*] Starting Last Byte Oracle...")
# 1024位 / 8位每字节 = 128次迭代
for k in range(1, 130):
    # 构造密文使明文扩大 256^k 倍
    target_c = (c_flag * pow(256**k, e, n)) % n

    # 这里的 decrypt 会调用 disguise，最后一个字节是原字节
    res_disguised = decrypt(target_c)
    if not res_disguised:
        last_byte = 0
    else:
        last_byte = res_disguised[-1]

    # 计算这一步减去了多少个 n (商 q)
    # 256 * m_prev = q*n + m_curr => m_curr = -q*n (mod 256)
    q = ((-last_byte) * n_inv) % 256

    # 收缩范围
    diff = high - low
    new_low = low + (q * diff) // 256
    new_high = low + ((q + 1) * diff) // 256
    low, high = new_low, new_high

    if k % 20 == 0:
        # 打印当前最高位可能的字符串
        print(f"Iter {k}: {long_to_bytes(high)}")

# 最终结果
flag = long_to_bytes(high)
print(f"\n[!] Flag: {flag}")
```

`hgame{E2r54_lS_StI1l_PrettY-e2,rlgHt?937315}`

ezDLP

首先对模数nn进行质因数分解，利用行列式性质

```
det(Ak)=(detA)k≡detB(modp)det(Ak)=(detA)k≡detB(modp)
```

将原矩阵方程转化为简单的模幂方程；随后在各素因子的乘法群中求解标量离散对数得到kk的残余系，并通过CRT还原出隐藏的指数kk（即 AES 密钥的种子），最后利用 MD5 哈希派生密钥并解密

```
from sage.all import *
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
from Crypto.Util.number import long_to_bytes
import hashlib
from base64 import b64decode
import json
from urllib.request import urlopen

def get_factors_from_factordb(n):
    print(f"[*] 正在尝试从 FactorDB 查询 n 的因子...")
    try:
        url = f"http://factordb.com/api?query={n}"
        with urlopen(url, timeout=15) as response:
            data = json.loads(response.read().decode())

        factors = []
        for f_str, count in data.get('factors', []):
            p = int(f_str)
            for _ in range(count):
                factors.append(p)

        prod = 1
        for x in factors: prod *= x
        if prod == n:
            print("[+] 成功从 FactorDB 获取并验证因子！")
            return sorted(factors)
        else:
            return []
    except Exception as e:
        print(f"[-] FactorDB 查询失败: {e}")
        return []

def solve():
    print("加载数据...")
    try:
        data = load('data.sobj')
        n, a, b = data
        print(f"[+] n 加载成功 (位长: {n.nbits()})")
    except Exception as e:
        print(f"[-] 失败: {e}")
        return
```

```
    # 1. 分解
    print("-" * 60)
    factors_list = get_factors_from_factordb(n)
    if not factors_list:
        print("[!] 请手动分解 n 或检查网络。")
        # 如果网络失败，请在这里手动填入 factor(n) 的结果
        # factors_list = [p1, p2]
        return

    from collections import Counter
    factors_summary = list(Counter(factors_list).items())

    # 2. 求解 DLP (优先使用行列式法)
    print("-" * 60)
    print("步骤 2: 求解 DLP (Determinant Method)")

    k_remainders = []
    k_moduli = []

    for p_val, exponent in factors_summary:
        p = Integer(p_val)
        print(f"\n[*] 处理素因子 p (位长: {p.nbits()})...")

        try:
            F = GF(p)
            # 计算行列式
            det_A = F(a.change_ring(F).det())
            det_B = F(b.change_ring(F).det())

            print(f"    det(A) = {det_A}")
            print(f"    det(B) = {det_B}")

            if det_A == 0:
                print("    [-] det(A) 为 0，无法使用此方法。")
                continue
            if det_A == 1:
                print("    [-] det(A) 为 1，无法提取信息 (k 可以是任意值)。")
                # 如果行列式法失败（阶为1），才尝试特征值法
                # 但这里为了简洁，先假设行列式有效
                continue

            # 求解 DLP: det(A)^k = det(B) mod p
            # 这实际上是在群 GF(p)* 中求解
            order = det_A.multiplicative_order()
            print(f"    乘法阶 (Order): {order}")
            print(f"    Order 位长: {order.nbits()}")

            print("    正在计算 discrete_log (行列式法)...")
            # 这是一个标量 DLP，比矩阵 DLP 快得多
            k_val = discrete_log(det_B, det_A, order)

            print(f"    [+] 成功! k = {k_val} (mod {order})")
            k_remainders.append(k_val)
```

```
            k_moduli.append(order)

        except Exception as e:
            print(f"    [-] 发生错误: {e}")
            print("    [!] 尝试备用方案: 特征值法 (仅在基域)...")
            # 备用：之前的特征值法（仅当结果在基域时）
            try:
                F = GF(p)
                Ap = a.change_ring(F)
                Bp = b.change_ring(F)
                lam = Ap.eigenvalues()[0]
                # 寻找对应的特征向量略... 简化处理，直接假设第一个特征值有效
                # 需要重新匹配 lam^k = mu
                # 这里略过复杂实现，因为行列式法通常对随机矩阵都有效
            except:
                pass

    if not k_remainders:
        print("\n[-] 失败: 未能恢复 k。")
        return

    # 3. CRT 合并
    print("-" * 60)
    try:
        k_base = crt(k_remainders, k_moduli)
        k_period = lcm(k_moduli)
        print(f"[+] CRT 合并完成。")
        print(f"    k_base = {k_base}")
        print(f"    模数 (Period) 位长: {k_period.nbits()}")
    except Exception as e:
        print(f"[-] CRT 错误: {e}")
        return

    # 4. 解密
    print("-" * 60)
    print("[*] 正在搜索 k 并解密...")

    ciphertext =
b64decode("ieJNk5335o9lCy6Ar2XymrDy+HVHcQhikluNSra0kBafw1WDCyyuNPkLACeBsavy")

    # 模数通常是 (p1-1)/2 * (p2-1)/2 约等于 n/4，大约 1072 位
    # k 是 1000 位。
    # 所以 k = k_base 或 k = k_base + k_period (如果 k_base 很小)

    candidates = [k_base]
    # 如果 CRT 结果比 k_period 小很多，多加几个周期试试
    for i in range(5):
        candidates.append(k_base + (i+1) * k_period)

    found = False
    for cand in candidates:
        if cand.nbits() < 950: continue
```

```
        print(f"    Checking k candidate (bits: {cand.nbits()})...")

        if is_prime(cand):
            try:
                key = hashlib.md5(long_to_bytes(int(cand))).digest()
                cipher = AES.new(key, AES.MODE_ECB)
                pt = cipher.decrypt(ciphertext)

                if b"flag" in pt or b"{" in pt:
                    try:
                        flag = unpad(pt, AES.block_size)
                        print("\n" + "#"*50)
                        print(f"FLAG: {flag.decode()}")
                        print("#"*50 + "\n")
                        found = True
                        break
                    except:
                        pass
            except Exception:
                continue

    if not found:
        print("[-] 未自动找到 Flag。请检查 k_base 是否正确。")

if __name__ == '__main__':
    solve()
```

`hgame{1s_m@trix_d1p_rEal1y_sImpLe??}`

Decision

利用LLL，检查加密噪声并还原，sage跑一下

```
# sage -python solve.py
import ast
from sage.all import *

# 来自题目末尾注释（task.py 里给出的 q）
q = 25670862761254429982373322233104793369 7
n = 25
m = 15

def center(x):
    x = int(x) % q
    if x > q//2:
        x -= q
    return x

# 读 output
enc = ast.literal_eval(open("output", "rb").read().decode())
# enc: 200 blocks, each block: 15 tuples, each tuple len=26

def short_dual_y(samples):
```

```
    """
    samples: list of tuples length n+1, total M samples (e.g. 30)
    return: short integer vector y in Z^M s.t. A^T y = 0 (mod q)
    """
    M = len(samples)
    A = Matrix(GF(q), [s[:n] for s in samples])     # M x n over GF(q)
    AT = A.transpose()                              # n x M

    # kernel over GF(q)
    K = AT.right_kernel().basis_matrix()            # d x M, d ~= M-n
    # lift to ZZ
    KZ = Matrix(ZZ, [[center(x) for x in row] for row in K])

    # build lattice basis: span(KZ rows) + q*I_M
    B = KZ.stack(q * identity_matrix(ZZ, M))

    # LLL to get short vector
    L = IntegerLattice(B, lll_reduce=True)
    y = vector(ZZ, L.reduced_basis()[0])            # take shortest
    return y

def score(samples):
    """
    smaller score => more likely LWE
    """
    M = len(samples)
    b = [int(s[n]) for s in samples]
    y = short_dual_y(samples)
    t = sum(y[i] * b[i] for i in range(M)) % q
    return abs(center(t))

# ---------- Step A: 找一个 reference 块（一定是 bit=1 的那种） ----------
# 方法：扫描相邻块 i,i+1，合并后若 score 特别小，说明两块都为 1
best_i = None
best_sc = None
for i in range(199):
    sc = score(enc[i] + enc[i+1])   # 30 samples
    if best_sc is None or sc < best_sc:
        best_sc = sc
        best_i = i

ref = enc[best_i]  # 认为 ref 这块是 LWE(=1)
# 也可以用 enc[best_i+1]，都行

print("[+] reference index:", best_i, "best_sc:", best_sc)

# ---------- Step B: 用 reference 去判每个块 ----------
scores = [score(ref + enc[i]) for i in range(200)]
sorted_sc = sorted(scores)

# 通常会出现两簇，取一个“中间阈值”即可
# 这里给一个很实用的自动阈值：取第 100 个和第 101 个的几何平均
# （如果你的 1/0 比例偏斜，可以改分位点）
```

```
T = ZZ(sqrt(sorted_sc[99] * sorted_sc[100]))
print("[+] threshold T =", T)

bits = ['1' if s < T else '0' for s in scores]
bitstr = ''.join(bits)

# ---------- Step C: bits -> bytes（注意 little-endian） ----------
val = int(bitstr, 2)
msg = val.to_bytes(25, "little")
flag = b"hgame{" + msg + b"}"
print(flag)
```

eezzdlp

先恢复一下p，再求k，最后AES解密

```
from sage.all import *
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes
from Crypto.Util.Padding import unpad
import hashlib
from base64 import b64decode

# 1. 加载数据
try:
    data = load('data.sobj')
    n, a, b = data
    print(f"[+] 数据加载成功。n 位长: {n.nbits()}")
except:
    print("[-] 请确保 data.sobj 在当前目录下。")
    exit()

# 2. 分解 n (n = p^2)
p = Integer(n).isqrt()
assert p*p == n
print(f"[+] 成功恢复 p: {p.nbits()} bits")

# 3. 求解离散对数 k
# 利用行列式性质: det(B) = det(A)^k mod p^2
ga = a.determinant()
gb = b.determinant()

# 第一部分: k mod (p-1)
# 在 GF(p) 中求解
k_mod_p_minus_1 = discrete_log(mod(gb, p), mod(ga, p))
print(f"[+] 算出 k mod (p-1): {k_mod_p_minus_1}")

# 第二部分: k mod p
# 利用性质: (g^{p-1})^k = (g^k)^{p-1} mod p^2
# 令 G = g^{p-1}, H = h^{p-1}。在 mod p^2 下，这些值一定形如 1 + r*p
G = pow(ga, p-1, n)
H = pow(gb, p-1, n)
```

```
# 因为 G = (1 + r1*p), 所以 G^k = (1 + k*r1*p) = H mod p^2
r1 = (G - 1) // p
rh = (H - 1) // p
# k * r1 = rh mod p
k_mod_p = (rh * inverse_mod(r1, p)) % p
print(f"[+] 算出 k mod p: {k_mod_p}")

# 4. CRT 合并得到 k
k = crt([k_mod_p_minus_1, k_mod_p], [p-1, p])
print(f"[+] 成功恢复 k: {k}")

# 5. AES 解密
ciphertext = b64decode("Q3UBa1pz1fi35L94peaFbPvpQe4UyXOUif3CKS/CmZdXOiV7bA5NNNjJ1KeUiAFE")
key = hashlib.md5(long_to_bytes(int(k))).digest()
cipher = AES.new(key, AES.MODE_ECB)

try:
    decrypted = cipher.decrypt(ciphertext)
    flag = unpad(decrypted, AES.block_size)
    print("-" * 30)
    print(f"Flag: {flag.decode()}")
    print("-" * 30)
except Exception as e:
    print(f"[-] 解密失败，请检查数据。错误: {e}")
```

Reverse

Marionette

打开 ELF 后，去字符串窗口（`Shift+F12`）找 OK、NO。

查看交叉引用，跳到main函数逻辑

我们注意到一个`memcmp`的比较逻辑（149行）

```
if ( !memcmp(&s1, &unk_405010, 0x10u) )
  buf = 19279;
else
  buf = 20302;
```

在 `memcmp` 调用的前后看一下参数

```
.rodata:000000000040500D                 db    0
.rodata:000000000040500E                 db    0
.rodata:000000000040500F                 db    0
.rodata:0000000000405010 unk_405010      db  8Ch                 ; DATA XREF: main+27A↑o
.rodata:0000000000405011                 db 0ADh
.rodata:0000000000405012                 db 0B4h
.rodata:0000000000405013                 db  8Fh
.rodata:0000000000405014                 db 0EBh
.rodata:0000000000405015                 db 0FDh
.rodata:0000000000405016                 db  6Fh ; o
.rodata:0000000000405017                 db 0AEh
.rodata:0000000000405018                 db  86h
.rodata:0000000000405019                 db  60h ; `
.rodata:000000000040501A                 db 0ADh
.rodata:000000000040501B                 db  44h ; D
.rodata:000000000040501C                 db 0C3h
.rodata:000000000040501D                 db 0C7h
.rodata:000000000040501E                 db  5Ah ; Z
.rodata:000000000040501F                 db  31h ; 1
.rodata:0000000000405020 xmmword_405020  xmmword 19C44AF193BBE32214F8D7B137C095Ah
```

常量位于 `0x405010`，内容是：

```
8cadb48febfd6fae8660ad44c3c75a31
```

到这里可以确定：程序最终就是比较 16 字节块是否等于这个目标值。

我们尝试观察输入到中间态的变换。程序先把输入 hex 解析成 16 字节，再进入一段混淆函数。中间有大量 `int3`，但输入预处理本身是可读的。

在 IDA 文本视图里看这些位置附近的字节操作：

`0x401f47`

`0x40206e`

`0x402116`

```
.text:0000000000401F42 byte_401F42     db 0CCh                 ; DATA XREF: .rodata:0000000000405160↓o
.text:0000000000401F43 ; ---------------------------------------------------------------------------
.text:0000000000401F43                 mov     [rsi+8], al
.text:0000000000401F46                 int     3               ; Trap to Debugger
.text:0000000000401F47                 mov     al, [rsi+7]
.text:0000000000401F4A                 xor     al, [rsi+6]
.text:0000000000401F4D                 sub     rsp, 8
.text:0000000000401F51                 int     3               ; Trap to Debugger
.text:0000000000401F52                 retn
.text:0000000000401F52 ; ---------------------------------------------------------------------------
.text:0000000000401F53 unk_401F53      db 0CCh                 ; DATA XREF: .rodata:0000000000405168↓o
.text:0000000000401F54 ; ---------------------------------------------------------------------------
.text:0000000000401F54                 xor     r10d, r8d
.text:0000000000401F57                 xor     r8d, r8d
.text:0000000000401F5A                 nop
.text:0000000000401F5B                 int     3               ; Trap to Debugger
.text:0000000000401F5C                 sub     rsp, 8
.text:0000000000401F60                 int     3               ; Trap to Debugger
.text:0000000000401F61                 retn
.text:0000000000401F61 ; ---------------------------------------------------------------------------
.text:0000000000401F62 byte_401F62     db 0CCh                 ; DATA XREF: .rodata:0000000000405170↓o
.text:0000000000401F63                 db 4Dh, 8Dh, 9
.text:0000000000401F66                 db 90h
.text:0000000000401F67
.text:0000000000401F67 ; =============== S U B R O U T I N E =======================================
.text:0000000000401F67
.text:0000000000401F67
.text:0000000000401F67 sub_401F67      proc near
.text:0000000000401F67                 xor     r8d, r8d
.text:0000000000401F6A                 sub     rsp, 8
.text:0000000000401F6E                 int     3               ; Trap to Debugger
.text:0000000000401F6F                 retn
.text:0000000000401F6F sub_401F67      endp ; sp-analysis failed
.text:0000000000401F6F
.text:0000000000401F70
```

这几段都会出现相同的模式：先取相邻两个字节，对它们进行异或，把结果写回后一个位置。

把这些片段连起来看，可以写成统一关系式。设原输入为 `x[0..15]`，进入后续加密前的中间值为 `y[0..15]`，则有：

`y0 = x0`
`yi = x(i-1) ^ xi`，其中 `i = 1..15`

这就是题面里那句 “echoes of my own past” 的含义：

对自身的部分进行异或。

现在我们已经有了倒数第二步的结果：

```
de731351e2d67abce313173404344404
```

记这个 16 字节为 y。现在只要把y逆成x。

由上面的关系式可得公式：

```
x0 = y0
x1 = y0 ^ y1
x2 = y0 ^ y1 ^ y2
............
```

以此类推

写个简单脚本

```python
y = bytes.fromhex("de731351e2d67abce313173404344404")

acc = 0
x = bytearray()
for b in y:
    acc ^= b
    x.append(acc)

print(x.hex())

# deadbeef0ddba11dfeedfacecafebabe
```

得到最终结果

```
hgame{deadbeef0ddba11dfeedfacecafebabe}
```